

Functions and Lambdas

CIS531 — Fall 2025, Syracuse

Kristopher Micinski



Functions

- ◆ Functions, procedures, methods: all similar concepts
 - ◆ A named block of code with clearly-specified inputs/outputs
- ◆ Functions supported by ~all languages
 - ◆ C-style “functions” — don’t allow true first-class functions
 - ◆ Racket functions, i.e., closures
 - ◆ Allows us to implement higher-order functions w/ proper lexical scope
 - ◆ More expressive than C functions
 - ◆ `(define (h x) (lambda () x))` — return value from h **captures** x in an environment

Adding define to our language...

- ◆ We will add a new construct:
 - ◆ `(define (f x y ...) e)` — Function named `f` accepting a fixed set of arguments
 - ◆ Compute `e`, return its value as the function's result

```
(define (fac x)
  (if (eq? x 0) 1 (* x (fac (- x 1)))))
(fac (read))
```



```

type ::= Integer | Boolean | (Vector type+) | Void | (type* -> type)
cmp  ::= eq? | < | <= | > | >=
exp  ::= int | (read) | (- exp) | (+ exp exp) | (- exp exp)
        | var | (let ([var exp]) exp)
        | #t | #f | (and exp exp) | (or exp exp) | (not exp)
        | (cmp exp exp) | (if exp exp exp)
        | (vector exp+) | (vector-ref exp int)
        | (vector-set! exp int exp) | (void)
        | (exp exp*)
def  ::= (define (var [var: type]*):type exp)
R4  ::= (program info def* exp)

```

Figure 6.1: Syntax of R_4 , extending R_3 (Figure 5.2) with functions.

- ◆ x86-64 assembler has some useful instructions for us to use...
 - ◆ `callq` — allows us to call functions, directly or indirectly (via a pointer)
 - ◆ `leaq` — load effective address, allows us to load function pointers
- ◆ The `callq` instruction...
 - ◆ Pushes the return address (pointer to next instruction after call) to stack
 - ◆ Jumps to the function
 - ◆ Does **not** pop `%rbp`, or handle reset stack
 - ◆ Must do this ourselves before we exit
 - ◆ Can use leave instruction (`mov %rbp, %rsp; pop %rbp`)
- ◆ The `ret` instruction pops the stack into the instruction pointer—returning
 - ◆ Return value goes in `%rax`
 - ◆ First 6 arguments go in `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8`, and `%r9`

Time T0:

We're about to call callee

```
.globl callee
callee:
    pushq    %rbp
    movq     %rsp, %rbp
    subq     $16, %rsp      # alloc. locals (16-byte aligned stack)
    movq     %rdi, -8(%rbp) # save first parameter
    movq     -8(%rbp), %rax  # rax = x
    addq     $1, %rax       # rax = x + 1
    movq     %rax, -16(%rbp) # y = rax
    movl     -16(%rbp), %rax # return y in rax
    leave
    ret
```

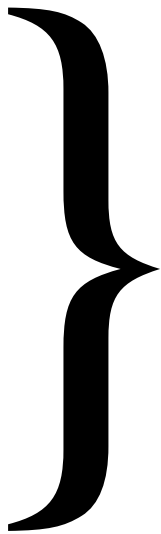
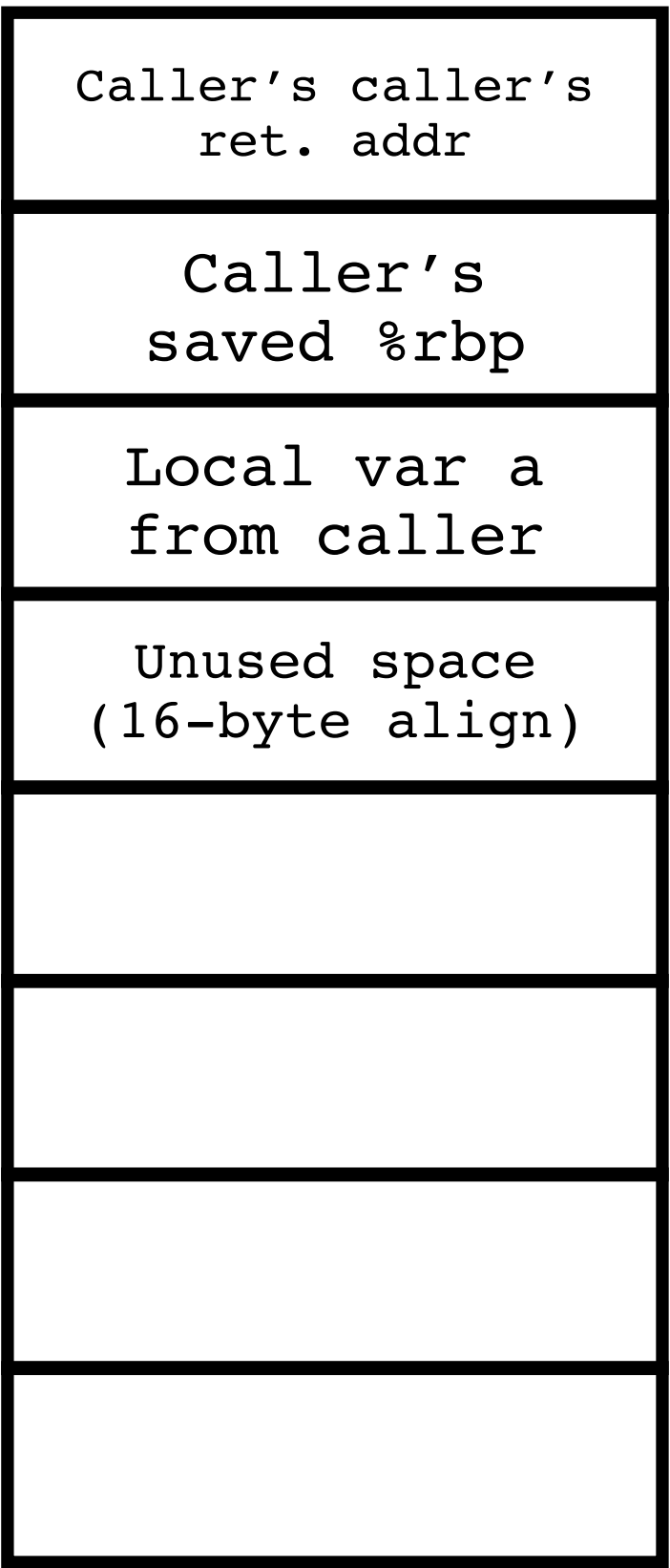
```
.globl caller
caller:
    pushq    %rbp
    movq     %rsp, %rbp
    subq     $16, %rsp      # space for locals
    movq     $5, -8(%rbp)   # int a = 5
    movq     -8(%rbp), %rdi # pass a in %rdi
    call     callee        # pushes return addr, jumps to callee
    movq     %rax, -16(%rbp) # b = return value
    movq     -16(%rbp), %rax # return b in %rax
    leave    # mov %rbp, %rsp; pop %rbp
    ret    # jump to return address on stack
```

%rip →

%rbp

%rbp-8

%rbp - 16 = %rsp →



caller's frame

%rsp points at the top of the stack
(But remember: stack grows **down**)

Time T1:

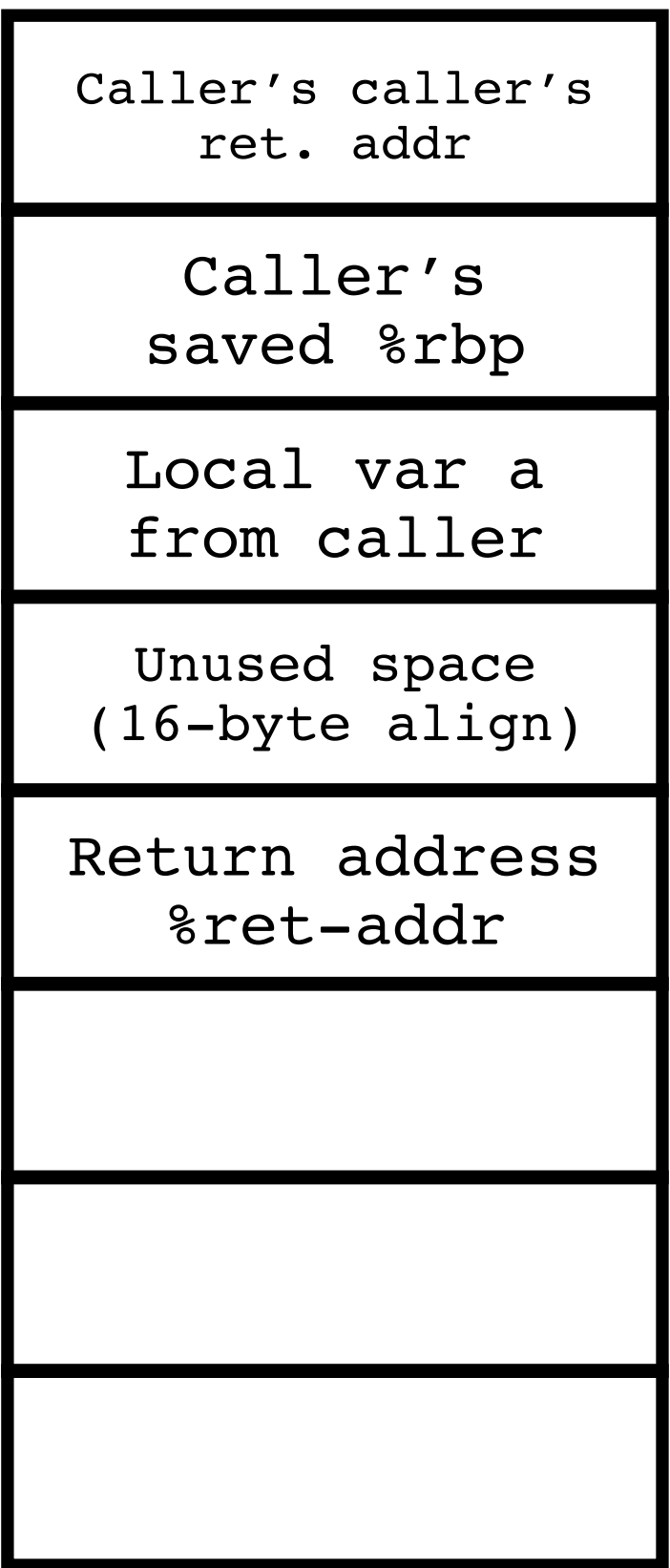
The call instruction is executed

```
.globl callee
callee:
    pushq    %rbp
    movq     %rsp, %rbp
    subq     $16, %rsp          # alloc. locals (16-byte aligned stack)
    movq     %rdi, -8(%rbp)    # save first parameter
    movq     -8(%rbp), %rax    # rax = x
    addq     $1, %rax          # rax = x + 1
    movq     %rax, -16(%rbp)   # y = rax
    movl     -16(%rbp), %rax    # return y in rax
    leave
    ret

.globl caller
caller:
    pushq    %rbp
    movq     %rsp, %rbp
    subq     $16, %rsp          # space for locals
    movq     $5, -8(%rbp)      # int a = 5
    movq     -8(%rbp), %rdi    # pass a in %rdi
    movq     %rdi, %rax        # pushes return addr, jumps to callee
    call     callee
    movq     %rax, -16(%rbp)    # b = return value
    movq     -16(%rbp), %rax    # return b in %rax
    leave
    ret                          # mov %rbp, %rsp; pop %rbp
                                # jump to return address on stack
```

%rip →
%ret-addr →

%rbp
%rbp-8
%rbp - 16
%rsp →



}
caller's frame

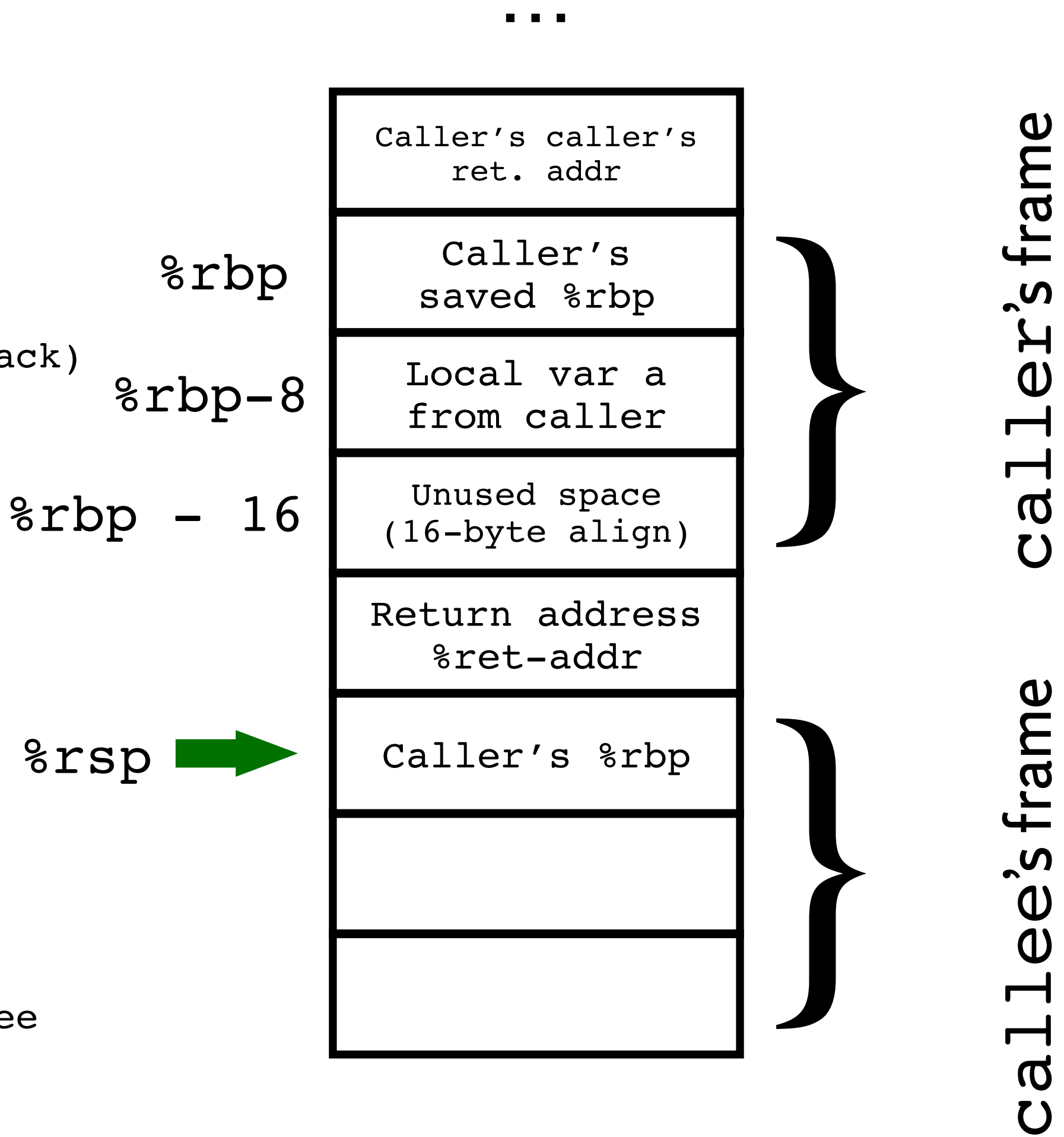
%rsp moves down, return address stored
%rbp retains its old value
%rip moves to beginning of callee

Time T2:

Callee starts executing...

```
.globl callee
callee:
%rip → pushq    %rbp
        movq    %rsp, %rbp
        subq    $16, %rsp      # alloc. locals (16-byte aligned stack)
        movq    %rdi, -8(%rbp) # save first parameter
        movq    -8(%rbp), %rax # rax = x
        addq    $1, %rax       # rax = x + 1
        movq    %rax, -16(%rbp) # y = rax
        movl    -16(%rbp), %rax # return y in rax
        leave
        ret

.globl caller
caller:
        pushq    %rbp
        movq    %rsp, %rbp
        subq    $16, %rsp      # space for locals
        movq    $5, -8(%rbp)   # int a = 5
        movq    -8(%rbp), %rdi # pass a in %rdi
        call    callee         # pushes return addr, jumps to callee
%ret-addr → movq    %rax, -16(%rbp) # b = return value
        movq    -16(%rbp), %rax # return b in %rax
        leave    # mov %rbp, %rsp; pop %rbp
        ret      # jump to return address on stack
```



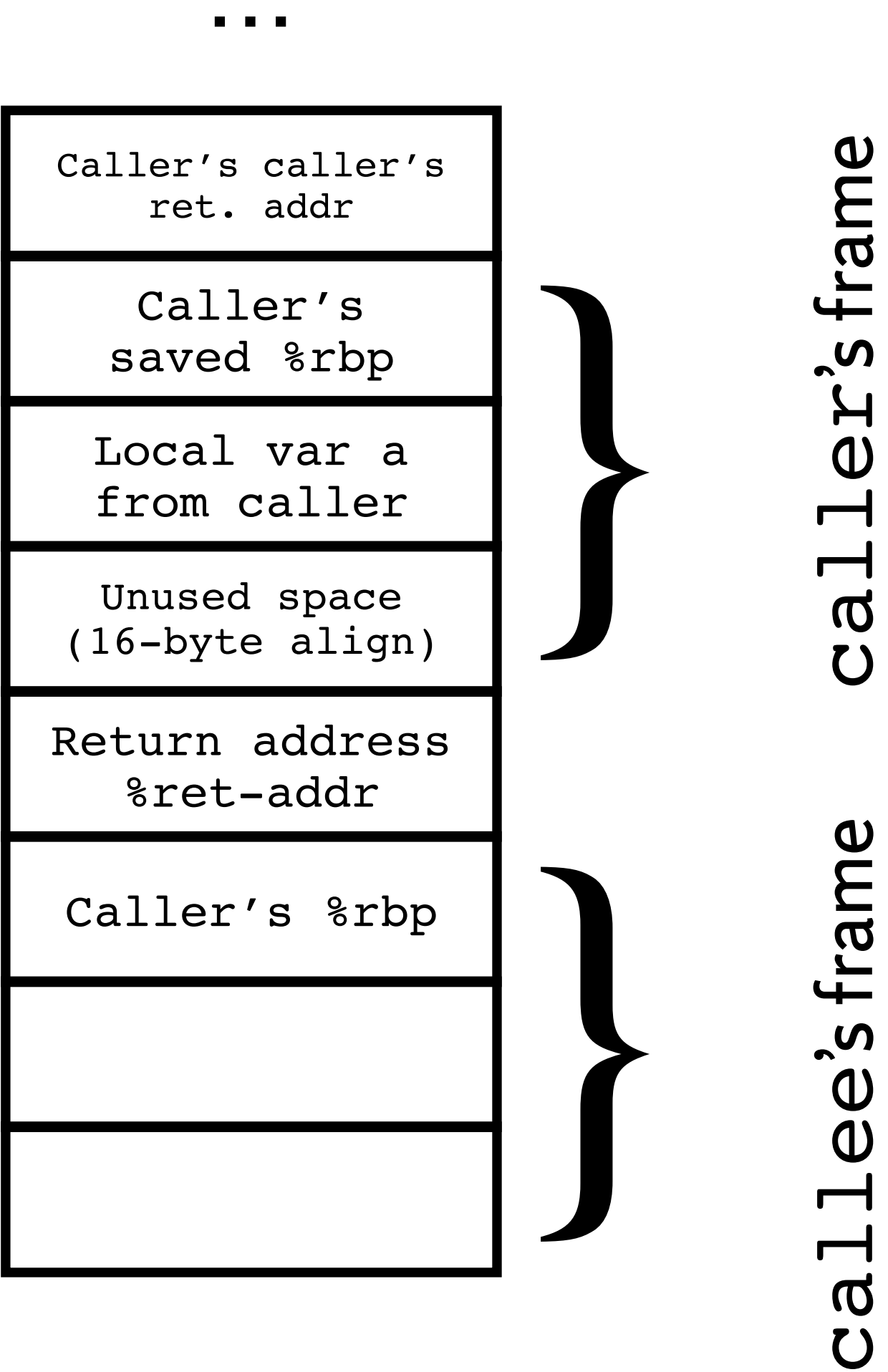
Executing the pushq instruction stores caller's %rbp (it is callee saved)

Time T3:

Establishing a new %rbp

```
.globl callee
callee:
    pushq    %rbp
    movq     %rsp, %rbp
    subq     $16, %rsp      # alloc. locals (16-byte aligned stack)
    movq     %rdi, -8(%rbp) # save first parameter
    movq     -8(%rbp), %rax  # rax = x
    addq     $1, %rax       # rax = x + 1
    movq     %rax, -16(%rbp) # y = rax
    movl     -16(%rbp), %rax # return y in rax
    leave
    ret

.globl caller
caller:
    pushq    %rbp
    movq     %rsp, %rbp
    subq     $16, %rsp      # space for locals
    movq     $5, -8(%rbp)   # int a = 5
    movq     -8(%rbp), %rdi # pass a in %rdi
    call     callee        # pushes return addr, jumps to callee
    movq     %rax, -16(%rbp) # b = return value
    movq     -16(%rbp), %rax # return b in %rax
    leave    # mov %rbp, %rsp; pop %rbp
    ret    # jump to return address on stack
```



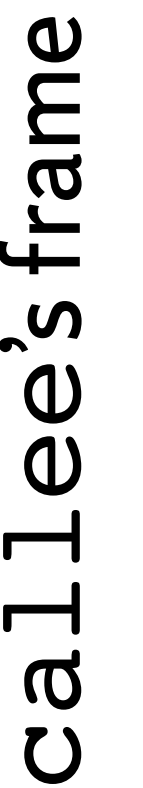
Notice how %rbp = %rsp is currently-used information, so we need subtract out space if we want to use it

The “first” free space is %rbp-8

Allocating stack space

ret

```
ret          # jump to return address on stack
```



Again, to keep stack 16-byte-aligned, we use unused space
(We don't need it in this case—no calls in this function!)

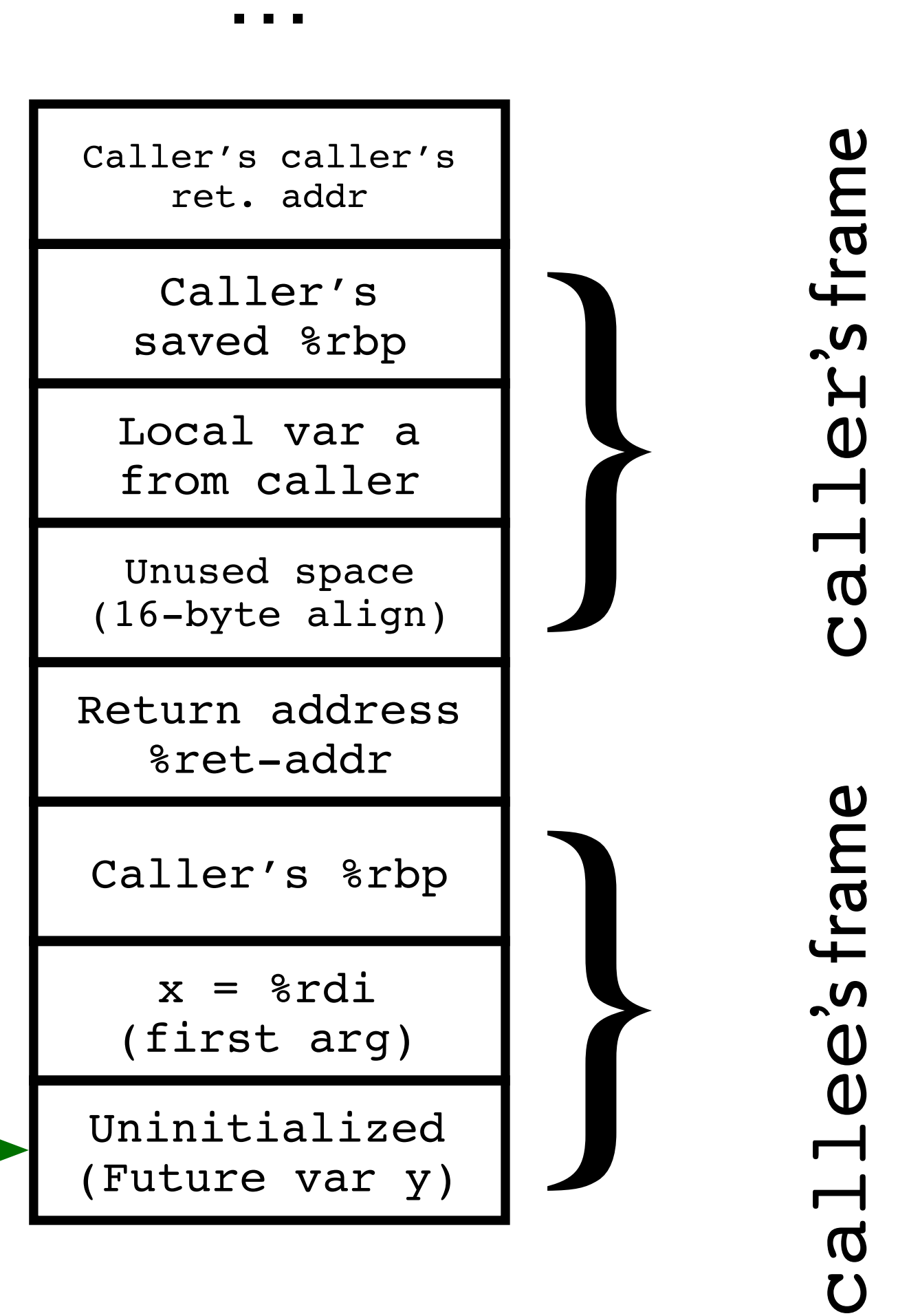
Moving first arg to local

```

.global callee
callee:
    pushq    %rbp                                %rbp
    movq     %rsp, %rbp
    subq     $16, %rsp                          # alloc. locals (16-byte aligned stack)
    movq     %rdi, -8(%rbp) # save first parameter %rbp-8
    movq     -8(%rbp), %rax # rax = x
    addq     $1, %rax                          # rax = x + 1
    movq     %rax, -16(%rbp) # y = rax %rbp - 16
    movl     -16(%rbp), %rax # return y in rax
    leave
    ret

.global caller
caller:
    pushq    %rbp                                %rbp
    movq     %rsp, %rbp
    subq     $16, %rsp                          # space for locals
    movq     $5, -8(%rbp) # int a = 5
    movq     -8(%rbp), %rdi # pass a in %rdi
    call     callee # pushes return addr, jumps to callee %rsp
    movq     %rax, -16(%rbp) # b = return value
    movq     -16(%rbp), %rax # return b in %rax
    leave
    ret
    # mov %rbp, %rsp; pop %rbp
    # jump to return address on stack

```



Move %rdi (holds first argument) into -8(%rbp), the space representing the variable x. Space for y was allocated but hasn't been initialized yet...

Skipping forward...

Time T8:

Moving %rax to y

```
.globl callee
callee:
    pushq    %rbp
    movq     %rsp, %rbp
    subq     $16, %rsp          # alloc. locals (16-byte aligned stack)
    movq     %rdi, -8(%rbp)     # save first parameter
    movq     -8(%rbp), %rax     # rax = x
    addq     $1, %rax           # rax = x + 1
    movq     %rax, -16(%rbp)    # y = rax
    movl     -16(%rbp), %rax    # return y in rax
    leave
    ret

.globl caller
caller:
    pushq    %rbp
    movq     %rsp, %rbp
    subq     $16, %rsp          # space for locals
    movq     $5, -8(%rbp)       # int a = 5
    movq     -8(%rbp), %rdi     # pass a in %rdi
    call     callee             # pushes return addr, jumps to callee
    movq     %rax, -16(%rbp)    # b = return value
    movq     -16(%rbp), %rax    # return b in %rax
    leave    # mov %rbp, %rsp; pop %rbp
    ret    # jump to return address on stack
```

%rip →

%ret-addr →

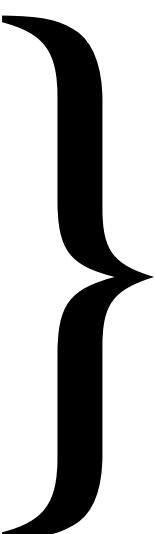
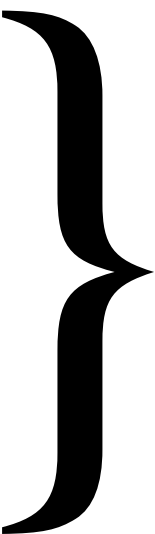
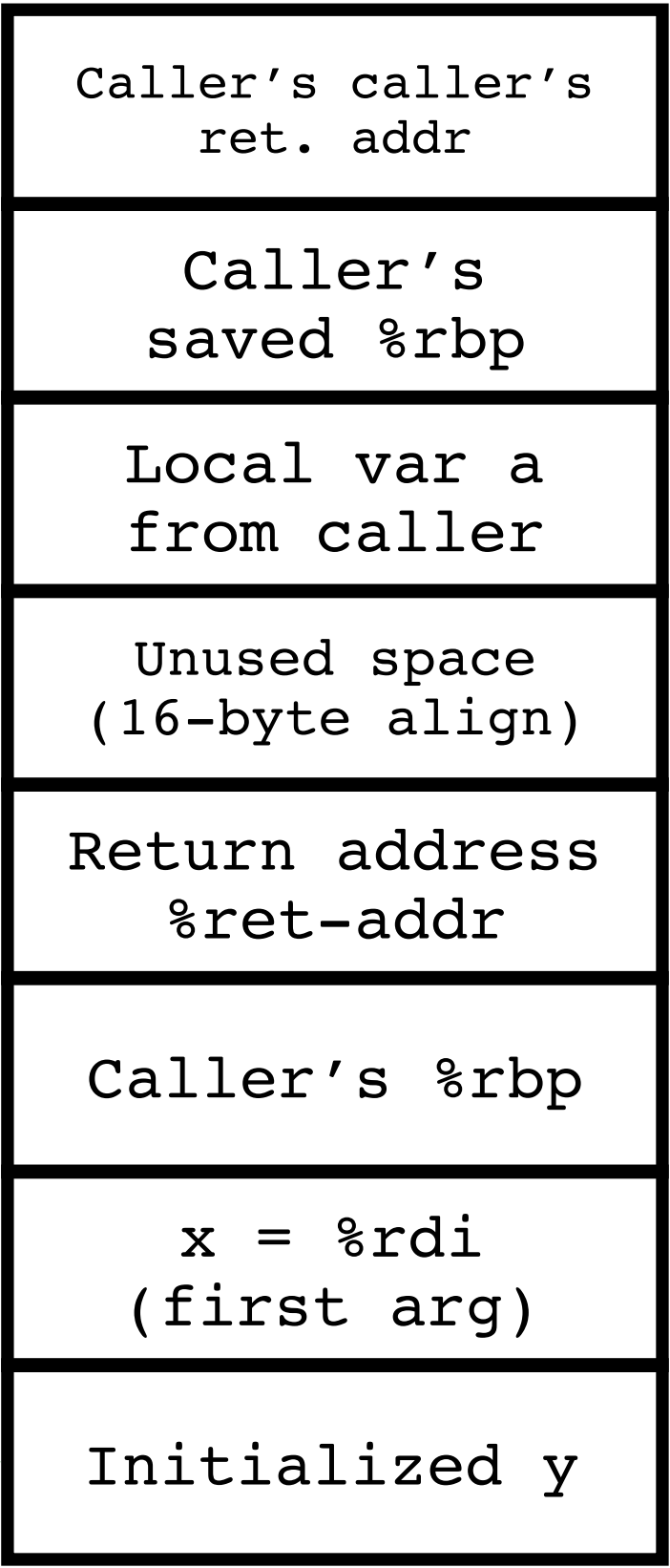
%rbp

%rbp-8

%rbp - 16

%rbp

%rsp →



caller's frame

callee's frame

Compute result into %rax, then store at -16(%rbp)

Time T9:

Copy back

```

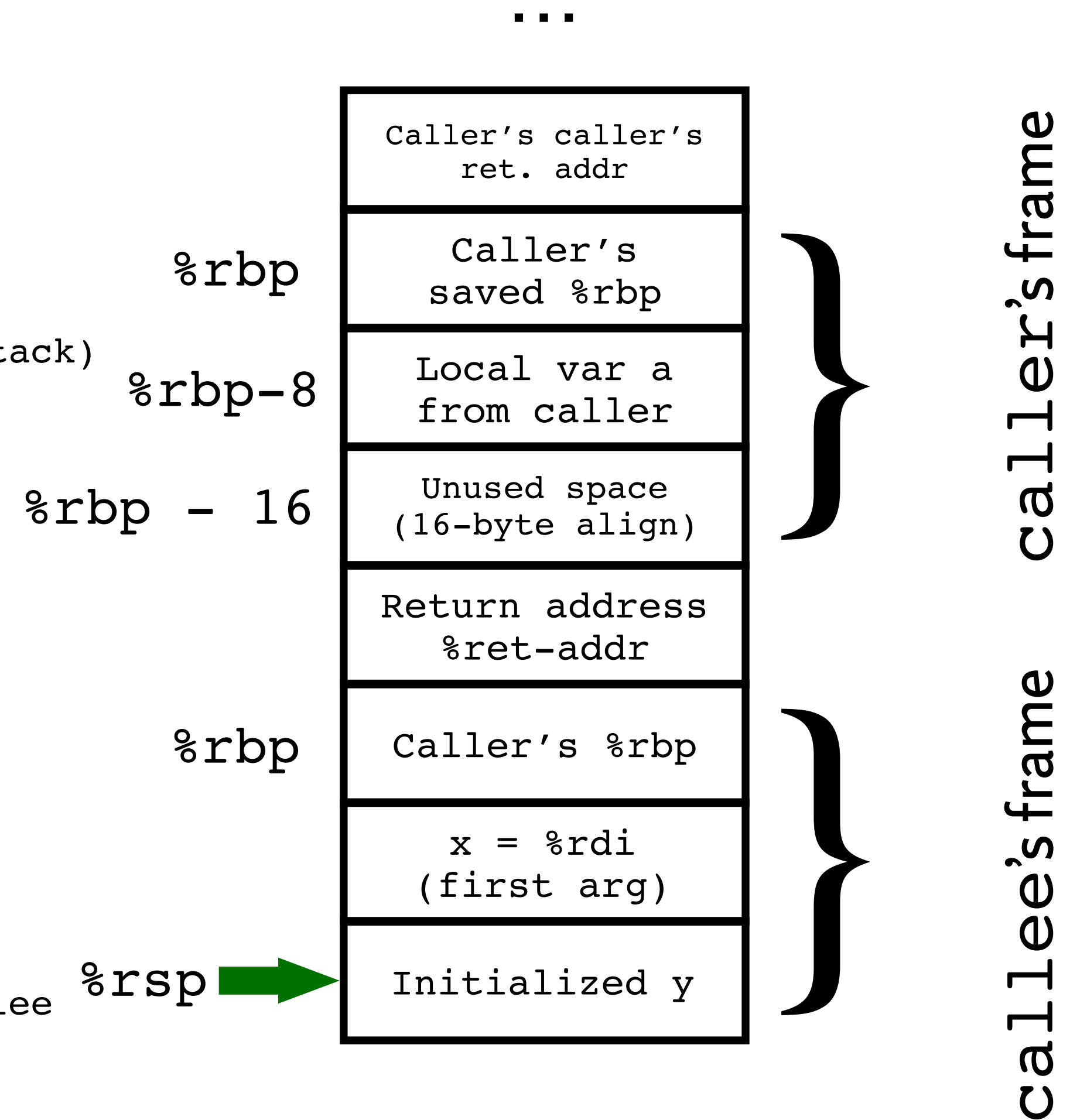
.global callee
callee:
    pushq    %rbp
    movq     %rsp, %rbp
    subq     $16, %rsp      # alloc. locals (16-byte aligned stack)
    movq     %rdi, -8(%rbp) # save first parameter
    movq     -8(%rbp), %rax  # rax = x
    addq     $1, %rax        # rax = x + 1
    movq     %rax, -16(%rbp) # y = rax
    movl     -16(%rbp), %eax # return y in rax
    leave
    ret

```

```

.global caller
caller:
    pushq    %rbp
    movq     %rsp, %rbp
    subq     $16, %rsp        # space for locals
    movq     $5, -8(%rbp)     # int a = 5
    movq     -8(%rbp), %rdi   # pass a in %rdi
    call     callee           # pushes return addr, jumps to callee
    movq     %rax, -16(%rbp)  # b = return value
    movq     -16(%rbp), %rax  # return b in %rax
    leave    # mov %rbp, %rsp; pop %rbp
    ret      # jump to return address on stack

```



Now copy it back into %rax (we might optimize this out later)

The leave instruction...

%rip →

`%ret-addr` 



Combo instruction: first does `mov %rbp into %rsp`, then pops into `%rbp`; we're ready to `ret`

Time T11:

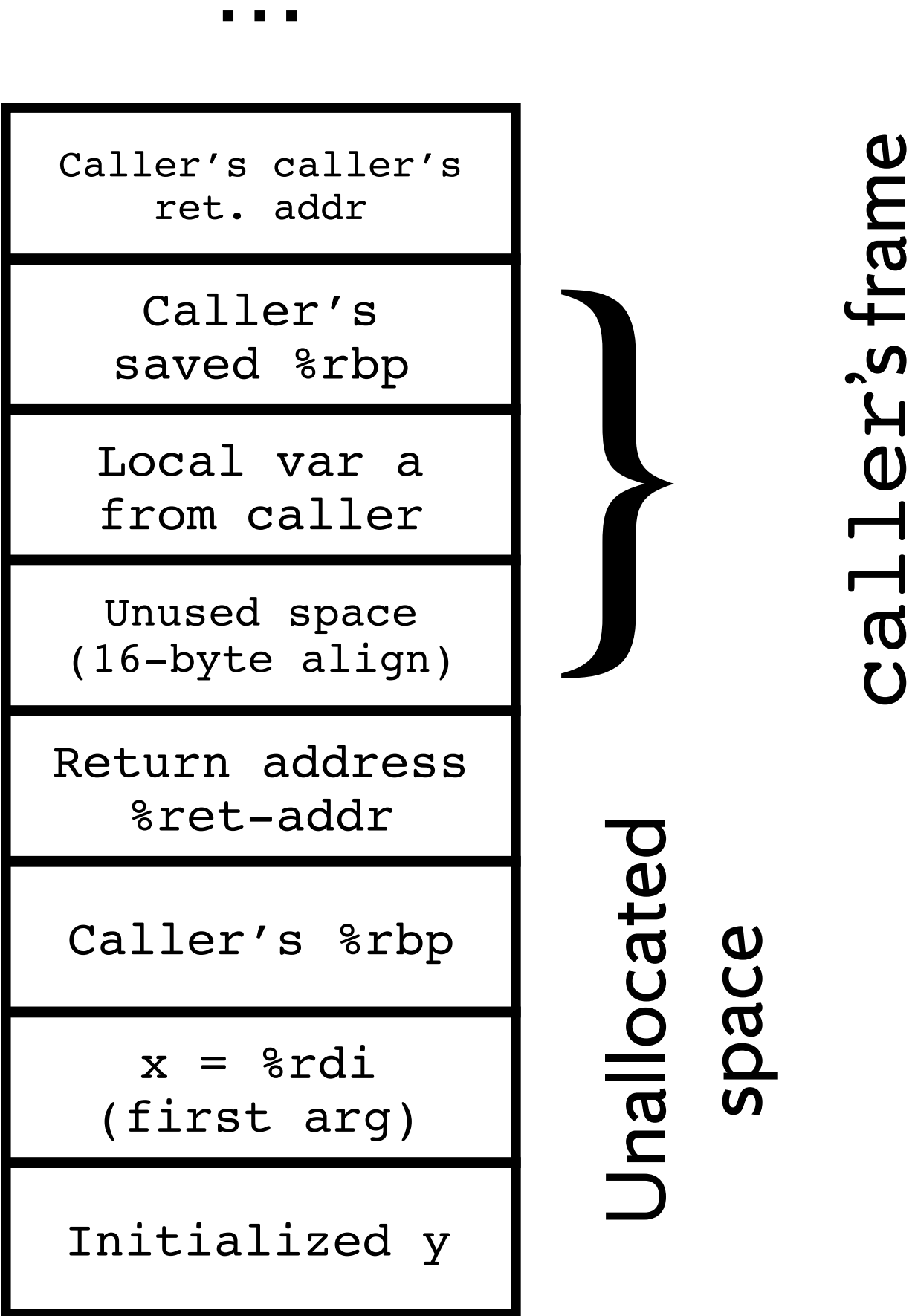
The ret instruction

```
.globl callee
callee:
    pushq    %rbp
    movq     %rsp, %rbp
    subq     $16, %rsp      # alloc. locals (16-byte aligned stack)
    movq     %rdi, -8(%rbp) # save first parameter
    movq     -8(%rbp), %rax  # rax = x
    addq     $1, %rax       # rax = x + 1
    movq     %rax, -16(%rbp) # y = rax
    movl     -16(%rbp), %rax # return y in rax
    leave
    ret
```

```
.globl caller
caller:
    pushq    %rbp
    movq     %rsp, %rbp
    subq     $16, %rsp      # space for locals
    movq     $5, -8(%rbp)   # int a = 5
    movq     -8(%rbp), %rdi # pass a in %rdi
    call     callee         # pushes return addr, jumps to callee
    movq     %rax, -16(%rbp) # b = return value
    movq     -16(%rbp), %rax # return b in %rax
    leave
    ret                    # mov %rbp, %rsp; pop %rbp
                          # jump to return address on stack
```

%rip

%rsp → %rbp - 16



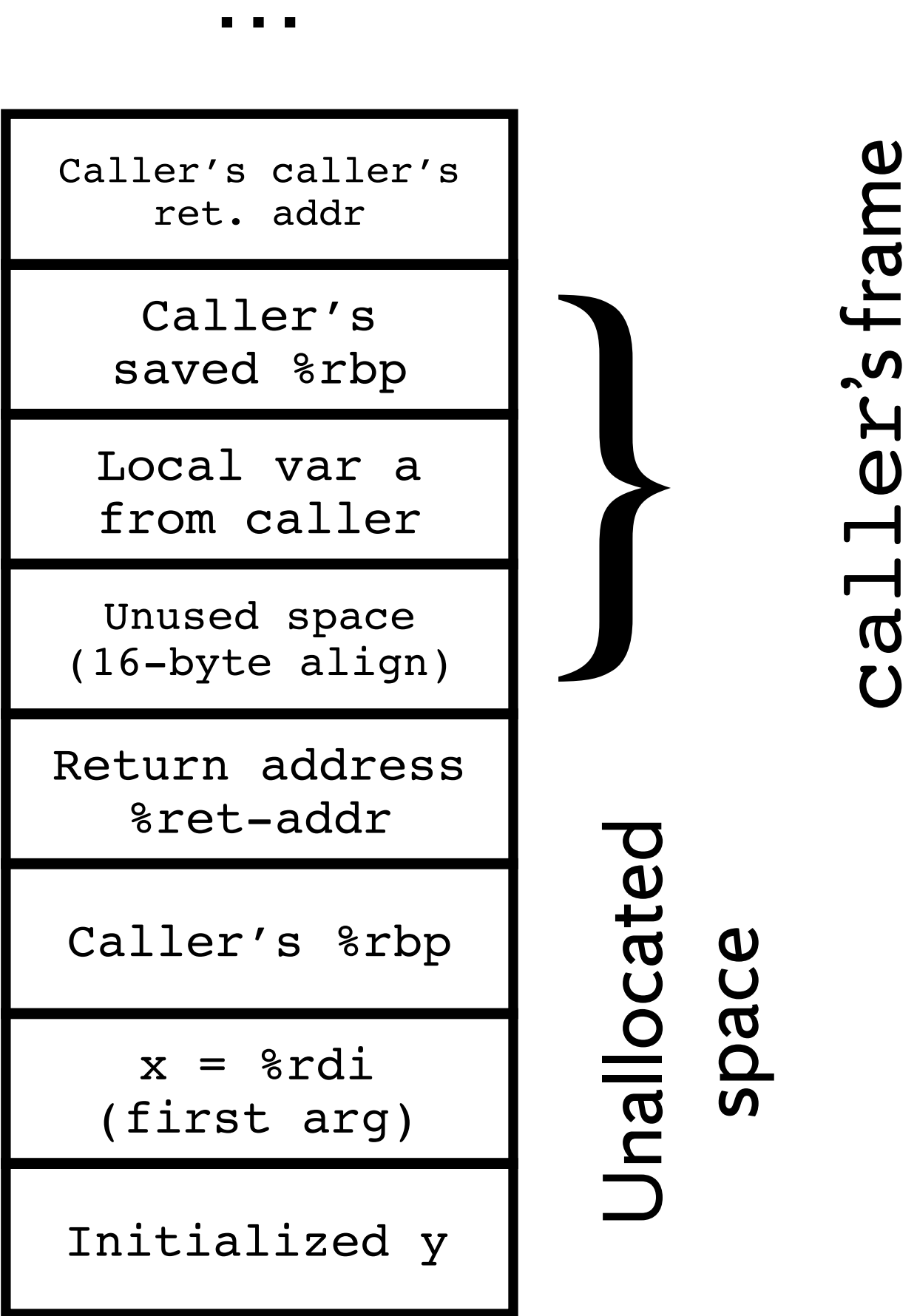
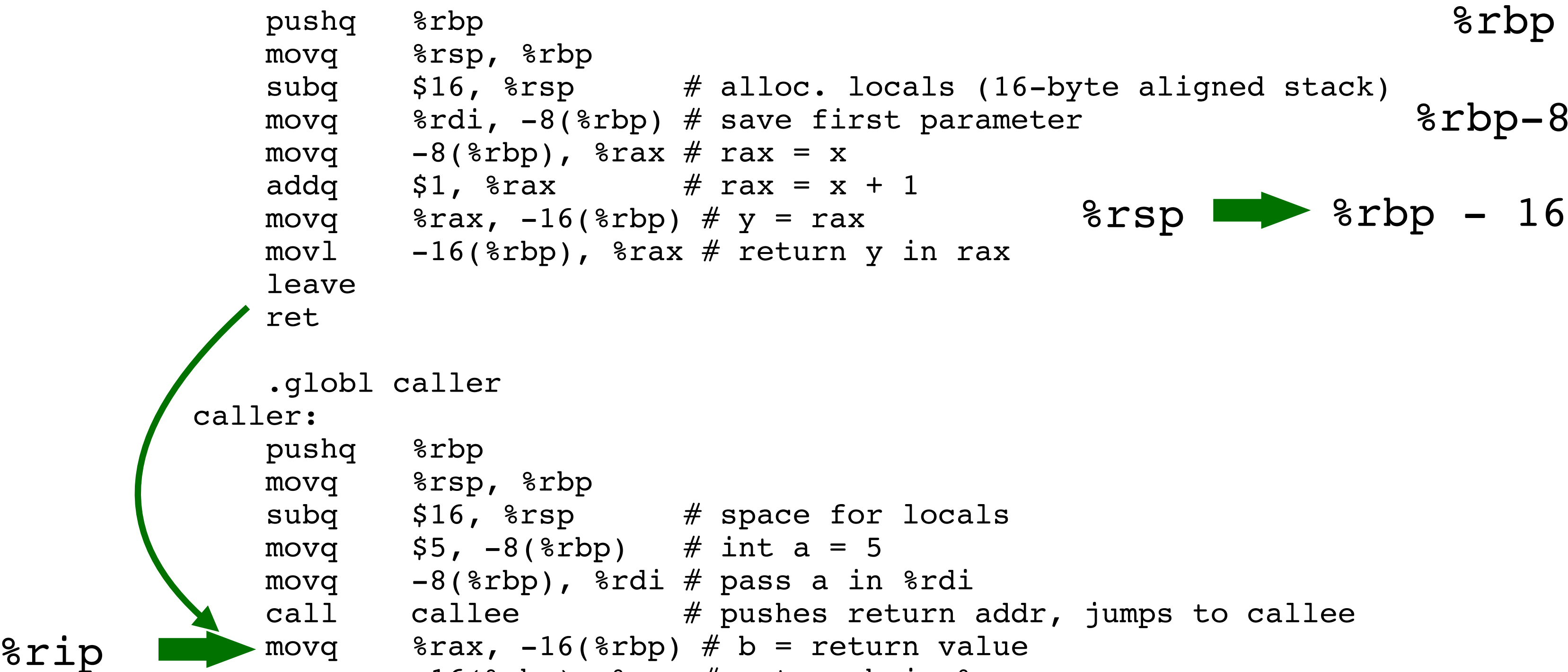
Now, we pop %rsp directly into %rip, which is %ret-addr
Past the bottom of the stack, we have the previous frame's remnants which will be overwritten

Time T12:

Copying the result...

```
.globl callee
callee:
    pushq    %rbp
    movq     %rsp, %rbp
    subq     $16, %rsp      # alloc. locals (16-byte aligned stack)
    movq     %rdi, -8(%rbp) # save first parameter
    movq     -8(%rbp), %rax  # rax = x
    addq     $1, %rax       # rax = x + 1
    movq     %rax, -16(%rbp) # y = rax
    movl     -16(%rbp), %rax # return y in rax
    leave
    ret

.globl caller
caller:
    pushq    %rbp
    movq     %rsp, %rbp
    subq     $16, %rsp      # space for locals
    movq     $5, -8(%rbp)   # int a = 5
    movq     -8(%rbp), %rdi # pass a in %rdi
    call     callee        # pushes return addr, jumps to callee
    movq     %rax, -16(%rbp) # b = return value
    movq     -16(%rbp), %rax # return b in %rax
    leave    # mov %rbp, %rsp; pop %rbp
    ret    # jump to return address on stack
```



We copy the result (%rax) back into local var, then move it into **our** return value (another place for potential optimization...). Last, **we** return as well,...

Time T15:

Now caller is returning, too...

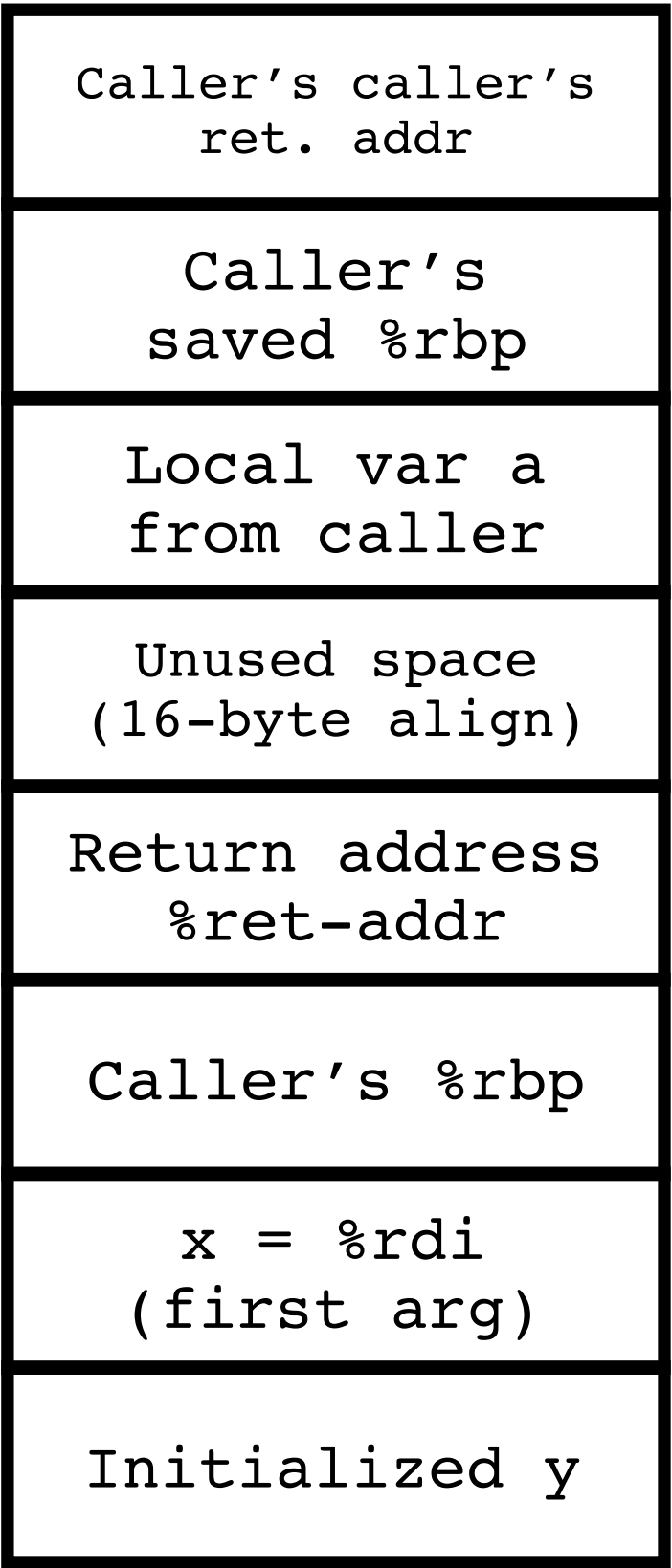
```
.globl callee
callee:
    pushq    %rbp
    movq     %rsp, %rbp
    subq     $16, %rsp      # alloc. locals (16-byte aligned stack)
    movq     %rdi, -8(%rbp) # save first parameter
    movq     -8(%rbp), %rax  # rax = x
    addq     $1, %rax       # rax = x + 1
    movq     %rax, -16(%rbp) # y = rax
    movl     -16(%rbp), %rax # return y in rax
    leave
    ret

.globl caller
caller:
    pushq    %rbp
    movq     %rsp, %rbp
    subq     $16, %rsp      # space for locals
    movq     $5, -8(%rbp)   # int a = 5
    movq     -8(%rbp), %rdi # pass a in %rdi
    call     callee        # pushes return addr, jumps to callee
    movq     %rax, -16(%rbp) # b = return value
    movq     -16(%rbp), %rax # return b in %rax
    leave    # mov %rbp, %rsp; pop %rbp
    ret    # jump to return address on stack
```

%rip →

%rbp →

%rsp →



Unallocated space
Unallocated space

Now we reinstate the caller's caller's %rbp, and we return too

Shrink pass and reveal-functions pass

- ◆ Our programs will be a sequence of top-level defines, followed by a main expression
 - ◆ $(\text{program } (\text{define } (f \dots) \dots) \dots e) \Rightarrow (\text{program } \dots (\text{define } (\text{main}) e))$
- ◆ Otherwise, apply to each expression
- ◆ It is helpful to have functions tagged explicitly:
 - ◆ $((\text{fun-ref } f) \ x \ y)$ vs $(+ \ x \ y)$ — book uses reveal-functions pass to explicate functions

Typechecking

- ◆ We will largely skip type checking (at least in the required projects)
 - ◆ Ensure that number of arguments matches call/return
 - ◆ Types of individual arguments match

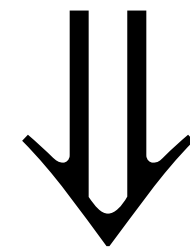
Implementing function calls...

- ◆ After ANF conversion, function calls in our language look like this:
 - ◆ `(let ([x (f a0 a1 ...)]) e-b)`
 - ◆ This is a direct-style call, and we will call `f` using `callq *%rbp`
 - ◆ In general, won't know `f` by name (may be result of another function)
- ◆ We also have tail calls:
 - ◆ `(f a0 a1 ...)`
 - ◆ Possible to implement tail calls more efficiently: rather than using `callq`, use `jmp`!
 - ◆ Need to ensure we “clean up” the stack frame (reinststate `%rbp`, reset stack, ...)

Issue: functions w/ >6 arguments

- ◆ What if more than 6 arguments?
 - ◆ Conventional option: pass them on stack
 - ◆ Challenges tail calls, however..
 - ◆ Another option (we take): rest of arguments go in a vector
 - ◆ Fix up function to immediately unpack vector into arguments
 - ◆ Thus, all arguments always in registers
- ◆ In x86-64, we pass only the first 6 arguments in registers
 - ◆ In this class, we'll also implement closures—so the first argument is an environment
 - ◆ Thus, one fewer argument for registers (unless we can optimize it...)


```
(define (f a0 a1 a2 a3 a4 a5 a6 a8 a9 a10)
  ...)
(f 1 2 3 4 5 6 7 8 9 10 11)
```



```
(define (f a0 a1 a2 a3 a4 a5 vec-rest)
  (let ([a6 (vector-ref vec-rest 0)]
        [a7 (vector-ref vec-rest 1)]
        [a8 (vector-ref vec-rest 2)]
        [a9 (vector-ref vec-rest 3)]
        [a10 (vector-ref vec-rest 4)]))
    ...))
(f 1 2 3 4 5 6 (vector 7 8 9 10 11))
```

Generating instructions for function calls...

- ◆ To translate a direct-style call, like (let ([x (f a0 a1 ...)]) e-b)
 - ◆ F's address loaded into %rax via leaq (or some other means...)
 - ◆ Load rest of args into registers (already turned into vector if >6)
 - ◆ After call, movq %rax into x
- ◆ To implement tail calls... (f a0 a1 ...)
 - ◆ Load all args into proper registers
 - ◆ F's address loaded into %rax
 - ◆ Clean up stack—can't do this yet (stack frame not laid out)—emit (tail-jump ...)
 - ◆ Do jmp *%rax

C3 language—adds functions

- fun-ref and call can be used as expressions
 - Direct-style function calls w/ atomic args
- New tailcall addition
- Adds defines, assume a main

```
arg ::= int | var | #t | #f
cmp ::= eq? | <
exp ::= arg | (read) | (- arg) | (+ arg arg) | (not arg) | (cmp arg arg)
      | (allocate int type) | (vector-ref arg int)
      | (vector-set! arg int arg) | (global-value name) | (void)
      | (fun-ref label) | (call arg arg*)
stmt ::= (assign var exp) | (return exp) | (collect int)
tail ::= (return exp) | (seq stmt tail)
      | (goto label) | (if (cmp arg arg) (goto label) (goto label))
      | (tailcall arg arg*)
def ::= (define (label [var:type]*) :type ((label . tail)+))
C3 ::= (program info def*)
```

Figure 6.6: The C_3 language, extending C_2 (Figure 5.11) with functions.

Instruction-like IR

- A few things added:
 - Indirect calls
 - Tail-jmp — bit of a hack to support jump + resetting stack (needs later IRs)
 - Leaq — load effective address, used with (fun-ref ...) to get function pointers
- We still have top-level procedures

```
arg ::= (int int) | (reg register) | (deref register int)  
      | (byte-reg register) | (global-value name)  
      | (fun-ref label)  
cc ::= e | l | le | g | ge  
instr ::= (addq arg arg) | (subq arg arg) | (negq arg) | (movq arg arg)  
          | (callq label) | (pushq arg) | (popq arg) | (retq)  
          | (xorq arg arg) | (cmpq arg arg) | (setcc arg)  
          | (movzbq arg arg) | (jmp label) | (jcc label) | (label label)  
          | (indirect-callq arg) | (tail-jmp arg)  
          | (leaq arg arg)  
block ::= (block info instr+)  
def ::= (define (label) info ((label . block)+))  
x863 ::= (program info def*)
```

Figure 6.7: The x86₃ language (extends x86₂ of Figure 5.13).

Racket's functions are “first class”

- ◆ In the sense that we can return them:
 - ◆ `(define (id x) x)`
 - ◆ `(define (f _) id)`
- ◆ Need to be able to return **functions** at runtime
- ◆ We extend our blocks (C-like) IR to allow referencing a function
 - ◆ At runtime, we use `leaq fun_name(%rip), %result` to load a function pointer

Notice this new layer, F1, which is used for translating functions—it introduces fun-ref

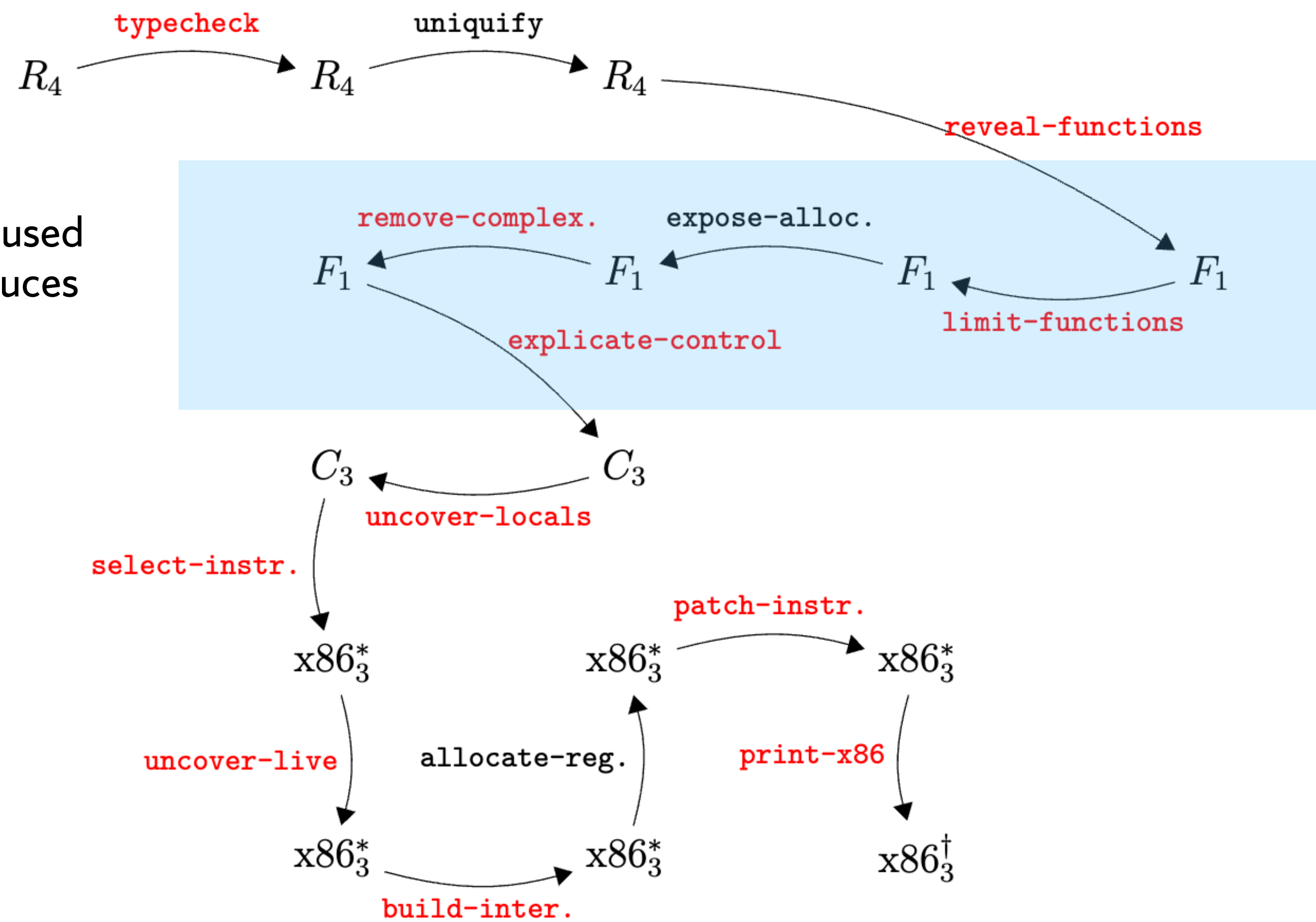


Figure 6.9: Diagram of the passes for R_4 , a language with functions.


```

type ::= Integer | Boolean | (Vector type+) | Void | (type* -> type)
exp  ::= int | (read) | (- exp) | (+ exp exp)
      | var | (let ([var exp]) exp)
      | #t | #f | (not exp) | (cmp exp exp) | (if exp exp exp)
      | (vector exp+) | (vector-ref exp int)
      | (vector-set! exp int exp) | (void) | (app exp exp*)
      | (fun-ref label)
def  ::= (define (label [var:type]*):type exp)
F1  ::= (program info def*)

```

Figure 6.5: The F_1 language, an extension of R_4 (Figure 6.1).

The “nested functions” problem...

```
(define (foo x y)
  (define (f z)
    (if (> y z)
        (+ x z)
        (- x z) ) )
  f )

(f (read) (read))
```

- * At the assembly level, I **can't** nest functions
 - * Variables are given a meaning via their lifetimes on the **stack**; once foo has returned, its stack frame is gone too, so x/y can't be referenced!
- * If we compile f as a block of code, how will it know the values of x/y when f's frame excludes x/y?
- * Solution: **closures**
 - * Every time I return a function, I return both the function's code and free variables
 - * We will use vectors for this

The answer: Closure Conversion

```
(define (foo x y)
  (define (f z)
    (if (> y z)
        (+ x z)
        (- x z)))
  f)
```

```
(define (lifted-f z env)
  (let* ([x (vector-ref env 0)]
        [y (vector-ref env 1)])
    (if (> y z)
        (+ x z)
        (- x z))))
```

```
(let ([clo (lifted-foo 10 20 (vector))])
  ((vector-ref clo 0) 5 (vector-ref clo 1)))
```

- * Idea: for each subordinate definition, I calculate a set of free variables
- * At each function definition, I extend my list of arguments to include an environment
- * Environment gives definitions of free variables minus the current function's arguments
- * Insert additional let-bindings to “unpack” the environment

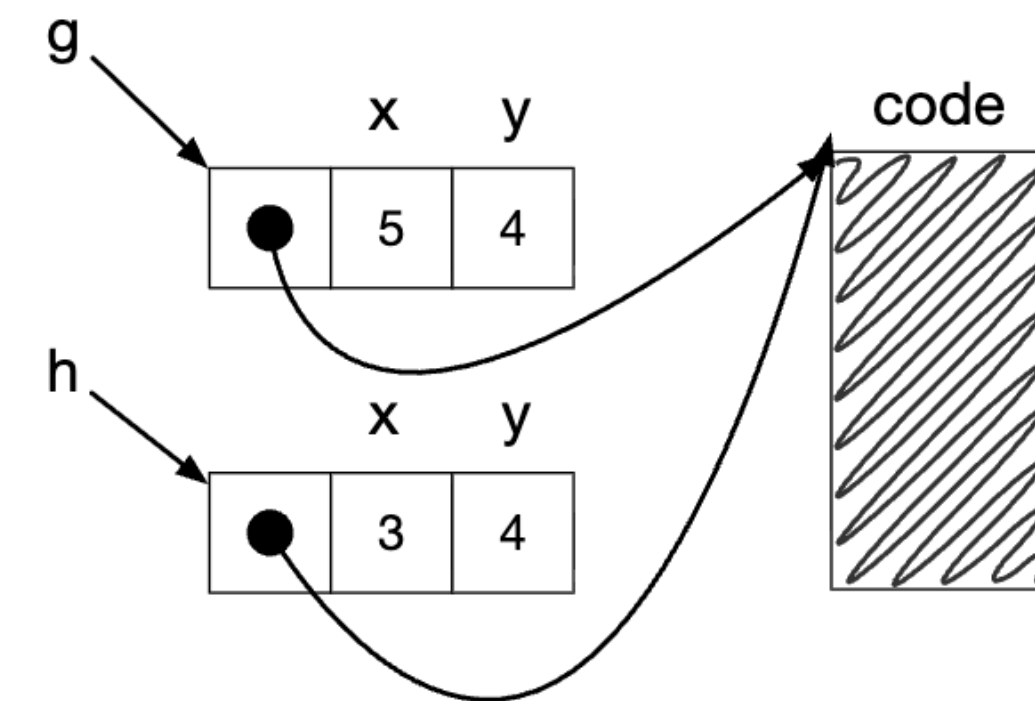
Implementing closures

- * Calculate free variables
 - * Establish some canonical ordering of freevars
- * Closures are vectors of size $n+1$ (n freevars)
 - * `data[0]` is a function pointer (to compiled code)
 - * `data[1]` holds the definition of the first free variable
 - * ...
 - * `data[n]` holds the last free variable
- * Every reference to a function name turns into a closure allocation: use canonical variable ordering to populate vector, then initialize to the proper function reference

```
(define (f x y)
  (λ (g) (g (+ x y))))
```

```
(define (λ-g g env)
  (let* ([x (vector-ref env 0)]
        [y (vector-ref env 1)])
    (g (+ x y))))
```

```
(define (f x y env) ;; env not used
  (vector λ-g (vector x y)))
```



<i>type</i>	$::=$	Integer Boolean (Vector $type^+$) Void ($type^* \rightarrow type$)
<i>exp</i>	$::=$	int (read) ($- exp$) ($+ exp exp$) ($- exp exp$) var (let ($[var exp]$) exp) $\#t$ $\#f$ (and $exp exp$) (or $exp exp$) (not exp) ($eq? exp exp$) (if $exp exp exp$) (vector exp^+) (vector-ref $exp int$) (vector-set! $exp int exp$) (void) ($exp exp^*$) (lambda: ($[var: type]^*$): $type exp$)
<i>def</i>	$::=$	(define ($var [var: type]^*$): $type exp$)
R_5	$::=$	(program $def^* exp$)

Figure 7.2: Syntax of R_5 , extending R_4 (Figure 6.1) with lambda.

Lambda Lifting

- * This is all the necessary machinery we need to implement **lambdas**
- * Code to do lambda lifting needs to “lift” lambda expression to a top-level define and give it a name
 - * Name then used when building lambda
- * Also takes an environment that unpacks free variables
 - * Uses a canonical variable ordering
- * Any syntactic lambda turns into a closure allocation
 - * Use canonical variable ordering to build closure
 - * Use vectors to build closures

```
(define (f x y)
  (λ (g) (g (+ x y))))
```

```
(define (λ-g g env)
  (let* ([x (vector-ref env 0)]
         [y (vector-ref env 1)])
    (g (+ x y))))
```

```
(define (f x y)
  (vector λ-g (vector x y)))
```


Example from book...

- * Reveal-functions adds fun-ref and makes app explicit
- * Closure conversion then turns lambdas into usages of (vector ...)
- * Adds administrative let binding to get function pointer
- * After closure conversion, the only new form we have is app
- * (In our projects we ignore type checking...)

```
(define (f [x : Integer]) : (Integer -> Integer)
  (let ([y 4])
    (lambda: ([z : Integer]) : Integer
      (+ x (+ y z))))))
(let ([g (f 5)])
  (let ([h (f 3)])
    (+ (g 11) (h 15)))))
```

⇓

```
(define (f (x : Integer)) : (Integer -> Integer)
  (let ((y 4))
    (lambda: ((z : Integer)) : Integer
      (+ x (+ y z)))))
(let ((g (app (fun-ref f) 5)))
  (let ((h (app (fun-ref f) 3)))
    (+ (app g 11) (app h 15)))))
```

⇓

```
(define (f (clos.1 : _) (x : Integer)) : (Integer -> Integer)
  (let ((y 4))
    (vector (fun-ref lam.1) x y)))
(define (lam.1 (clos.2 : _) (z : Integer)) : Integer
  (let ((x (vector-ref clos.2 1)))
    (let ((y (vector-ref clos.2 2)))
      (+ x (+ y z)))))
(let ((g (let ((t.1 (vector (fun-ref f))))
  (app (vector-ref t.1 0) t.1 5))))
  (let ((h (let ((t.2 (vector (fun-ref f))))
    (app (vector-ref t.2 0) t.2 3))))
    (+ (let ((t.3 g)) (app (vector-ref t.3 0) t.3 11))
      (let ((t.4 h)) (app (vector-ref t.4 0) t.4 15))))))
```

Figure 7.6: Example of closure conversion.

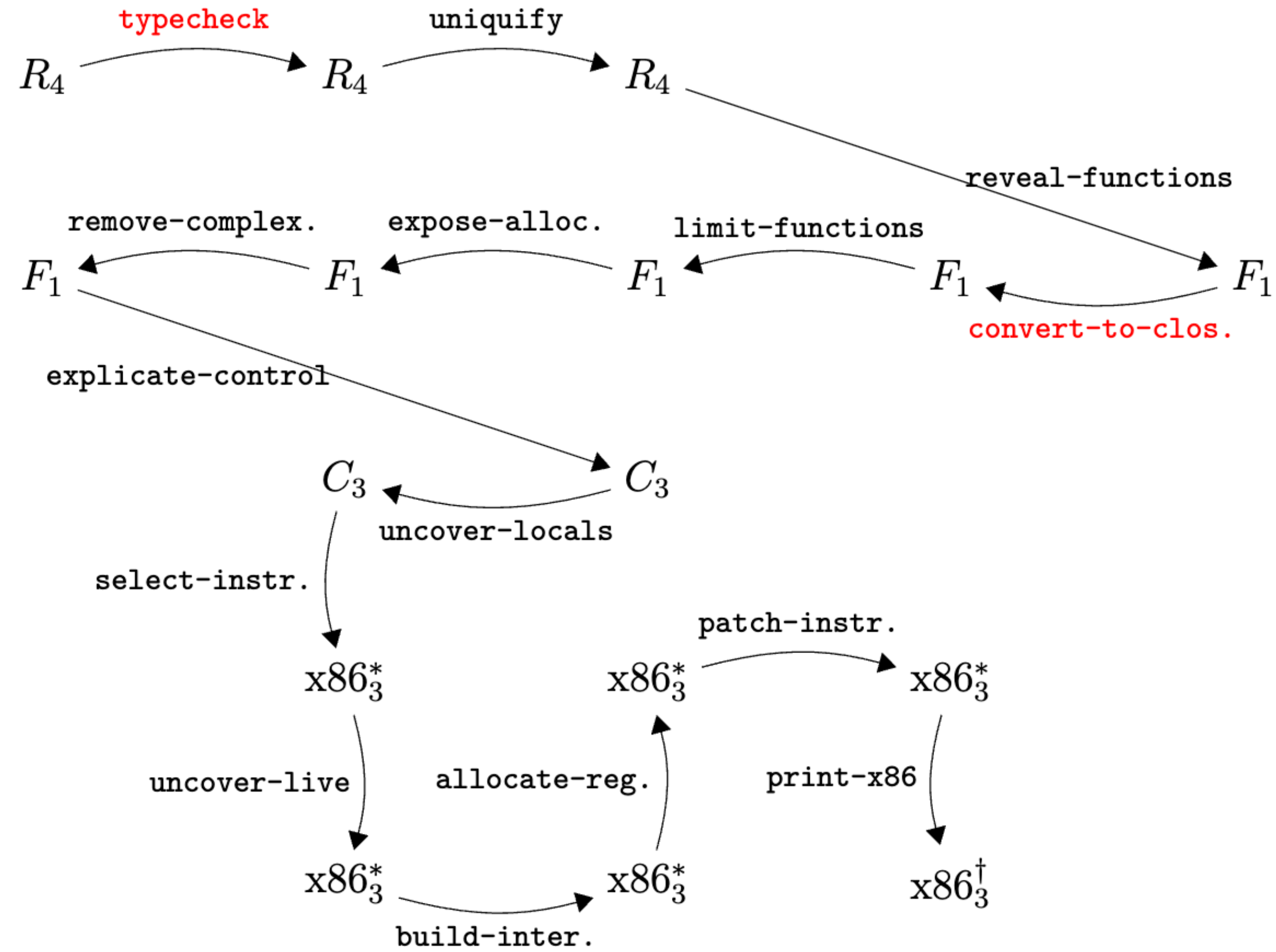


Figure 7.7: Diagram of the passes for R_5 , a language with lexically-scoped functions.

End of Required Project Material

- * This represents the end of the material, in terms of class projects:
 - * Project 4 covers vectors, set!, and loops
 - * Project 5 covers lambdas, closures, and functions
 - * Great accomplishment, we now have a fully-featured language!
- * So what next? Our language has serious limitations, where we could extend...
 - * Type systems: language is not type safe, has serious security vulnerabilities
 - * We will cover static/dynamic typing
 - * Register allocation: we will cover this in the coming weeks
 - * Necessary for a not-toy compiler, not too hard to implement
 - * Garbage collection / runtime system
 - * More builtin types (strings, cons cells, hash map, other core data structures)
 - * Match patterns / quasi quoting
 - * Program analysis / optimization: many places we're being slow!
 - * Exception handlers, continuations, and effects
- * You should pick **one** of these (or something else) for a final project